

Programare Orientată Pe Obiecte

Curs 1 - Introducere

Octombrie 2025

Agenda

- 1 Despre Curs
- 2 Paradigme de programare
- 3 Programarea orientată pe obiecte
- 4 Despre obiecte
- 5 Clase si instante
- 6 Despre JAVA

Despre Curs

Cadre didactice

Curs:

- Raluca Brehar
- raluca.brehar@cs.utcluj.ro
- Pagina cursului:
<https://moodle.cs.utcluj.ro/course/view.php?id=792>
- Teams -join with code **dmsbor8**

Laborator:

Conținutul cursului:

- Concepte și paradigme în programarea orientată pe obiecte
- Abstracțiuni și tipuri de date abstracte
- Caracteristicile limbajului Java
- Tipurile de date primitive și structurile de control în Java
- Clase și obiecte. Pachete
- Moștenirea și polimorfismul
- Interfețe Java
- Reprezentarea în UML a claselor și legăturilor dintre ele
- Excepții și tratarea lor
- Colecții Java
- Interfețe utilizator (GUIs) în Java
- Testarea și depanarea programelor
- Introducere în I/E Java
- Fire de lucru (threads) în Java

Prezență. Evaluare. Referințe

Evaluare:

- Nota finală = $0.4 * \text{Laborator} + 0.6 * \text{ExamenScriș}$
 - Obligatoriu Laborator ≥ 5 , Test1 - 40%, Test2 - 60%
 - Obligatoriu ExamenScriș ≥ 5
 - Optional – se poate realiza un proiect pentru maxim 1p în plus la nota finală de la laborator (temele de proiect se asignează în a doua jumătate a semestrului)

Referințe:

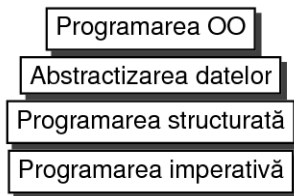
- Bruce Eckel, Thinking in Java, 4th edition, Prentice Hall, 2006
- Kathy Sierra, Bert Bates, SCJP Sun Certified Programmer for Java 6, Mc Graw Hill, 2008
- Tutoriale Java și Documentația Java de la Oracle
- Tutoriale UML introductive
- Documentația BlueJ, IneliJ Idea și Eclipse/Netbeans

Paradigme de programare

Paradigme de programare

Paradigma de programare:

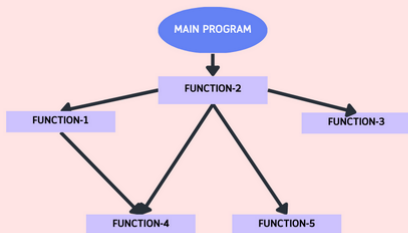
- Un model care descrie esența și structura computației
- Un stil fundamental de a programa
- Oferă (și determină) viziunea pe care o are programatorul asupra execuției programului
- Exemple:
 - în POO, programatorii pot concepe programele ca fiind o colecție de obiecte care interacționează
 - în programarea funcțională un program poate fi conceput ca fiind o secvență de evaluări de funcții, fără stări



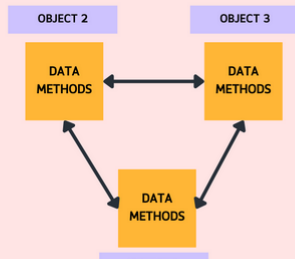
Programarea structurată vs programarea obiectuală

- Program procedural = definiții de date și apeluri de funcții/proceduri
- Program obiectual = abstractizarea informațiilor de comportament și stare din obiecte din lumea reală

Procedural Programming



Object Oriented Programming



Programarea structurată

Abstractizarea operațiilor

- Structura unui modul
 - Interfața: Date de intrare, Date de ieșire, Descrierea funcționalității
 - Implementarea: date locale, secvențe de instrucțiuni
- Sintaxa limbajului
 - Organizarea codului în blocuri de instrucțiuni
 - Definiții de funcții și proceduri
 - Extinderea limbajului cu noi operații
 - Apeluri la proceduri și funcții noi

Beneficiile Programării Structurate

Ușurează dezvoltarea software:

- Evită repetarea realizării aceluiași lucru
- Munca de programare este descompusă în module independente
- Proiectare Top-down: descompunerea în subprobleme

Facilitează întreținerea software:

- Codul este mai ușor de citit
- Independența modulelor

Favorizează reutilizarea software

```
int main()
{
    double u1, u2, m;
    u1 = 4;
    u2 = -2;
    m=sqrt (u1*u1 + u2*u2);
    printf("%lf\n", m);
    return 0;
}
```



```
double module(double u1, double u2)
{
    double m;
    m=sqrt (u1*u1 + u2*u2);
    return m;
}
int main()
{
    printf("%lf\n", module(4, -2));
    return 0;
}
```


Tip de date abstract în C

Exemplu

```
struct vector {  
    double x;  
    double y;  
}  
void construct (vector *v, double v1, double v2)  
{  
    v->x = v1;  
    v->y = v2;  
}  
double module(vector v)  
{  
    double m;  
    m = sqrt (v.x*v.x + v.y*v.y);  
    return m;  
}
```

```
int main()  
{  
    vector v;  
    construct(&v, 4, 2);  
    printf("%lf\n", module(v));  
    return 0;  
}
```

```
...  
double product(vector v, vector w) {  
    return v.x*w.x + v.y*w.y;  
}  
int main() {  
    vector v;  
    construct(&v, 4, 2);  
    construct(&w, -1, 7);  
    printf("%lf\n", product(v, w));  
    return 0;  
}
```

Beneficiile tipurilor de date abstracte

- Conceptele din domeniul real sunt reflectate în cod
- Încapsulare: complexitatea internă, datele și detaliile operațiilor sunt ascunse
- Utilizarea tipului de dată este independentă de implementarea sa internă
- Oferă o mai mare modularitate
- Sporește ușurința întreținerii și reutilizării codului

Exemplu

Se dorește lucrul cu dreptunghiuri. Un dreptunghi este definit prin lățime și înălțime.

- se citește de la tastatură lățimea și înălțimea,
- se calculează aria și perimetrul,
- se afișează rezultatele.

Scrieți codul C care rezolvă această problemă.

Exemplu

Rezolvare în C

```

1 double calculeazaAria(double latime, double inaltime) {
2     return latime * inaltime;
3 }
4 double calculeazaPerimetrul(double latime, double inaltime) {
5     return 2 * (latime + inaltime);
6 }
7 int main() {
8     double latime, inaltime;
9     printf("Introdu latimea: ");
10    scanf("%lf", &latime);
11    printf("Introdu inaltimea: ");
12    scanf("%lf", &inaltime);
13
14    double aria = calculeazaAria(latime, inaltime);
15    double perimetru = calculeazaPerimetrul(latime, inaltime);
16    printf("Aria: %.2lf\n", aria);
17    printf("Perimetrul: %.2lf\n", perimetru);
18    return 0;
19 }

```

Programarea orientată pe obiecte

Paradigma orientării pe obiecte

Paradigma programării structurate a avut inițial succes (1975-85)

- Dar a început să eșueze la produse mai mari ($> 50,000$ LOC)
- Avea probleme de întreținere post-livrare (astăzi această întreținere necesită, de la 70 la 80% din efortul total)
- Motivul: metodele structurate sunt fie:
 - orientate pe operații
 - orientate pe attribute
 - ... dar nu amândouă

Paradigma orientării pe obiecte

O simulare a domeniului unei probleme prin abstractizarea informațiilor de comportament și stare din obiecte din lumea reală

Conceptele POO

- Obiecte
- Clase
- Transmitere de mesaje
- Moștenire

POO consideră că atât atributele cât și operațiile au importanță egală

O viziune simplistă a unui obiect poate fi:

Obiect = componentă software care încorporează atât atributele cât și operațiile care se pot efectua asupra atributelor și care suportă moștenirea

Exemplu: obiect cont bancar

Date: soldul contului

Acțiuni: depune, retrage, determină soldul

Paradigma orientării pe obiecte

- 1 Totul este reprezentat prin obiecte (obiect = o variabilă mai specială ce încapsulează atât date cât și operații cu aceste date)
- 2 Obiectele comunică între ele prin trimitere/ primire de mesaje (mesaj = apel de metodă)
- 3 Obiectele au propria lor memorie
- 4 Orice obiect are un tip $\leftrightarrow$ orice obiect e o instanță a unei clase (unde 'clasa' este sinonim cu 'tip')
- 5 Toate obiectele de un anumit tip pot trimite sau primi aceleași mesaje

Punctele tari ale POO

Ascunderea informației $\Rightarrow$ întreținerea post-livrare este mai sigură

Șansele apariției erorilor regresive sunt reduse (în software nu se repetă erori cunoscute)

Dezvoltarea este mai ușoară

Obiectele au în general corespondente fizice $\Rightarrow$ Simplifică modelarea (un aspect cheie al POO)

Obiectele bine proiectate sunt unități independente

- Tot ce se referă la obiectul real modelat este în obiect $\rightarrow$ încapsulare
- Comunicarea se face prin schimb de mesaje
- Această independență promovează reutilizarea codului

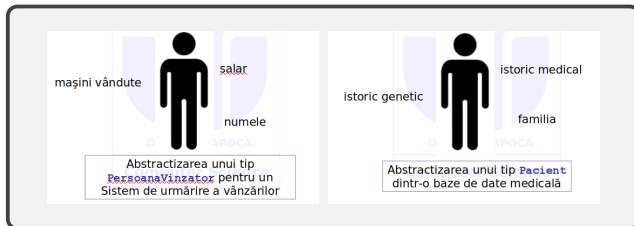
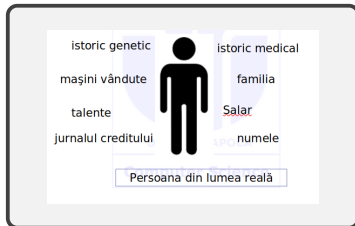
Programare orientată pe obiecte

Beneficii și concepte

- Oferă suport sintactic pentru tipurile de date abstracte și facilități asociate cu ierarhiile de clase
- Schimbă punctul de vedere: programele sunt apendice ale datelor
- Introduce un concept nou: **obiect = tip de dată abstract cu stare (atribute) și comportament (operații)**
- **Abstractizare** = proces de filtrare a detaliilor neimportante ale obiectului astfel încât să rămână doar caracteristicile (Ne ocupăm doar de datele care prezintă interes pentru problema noastră) importante

Abstractizare

Exemplu



Despre obiecte

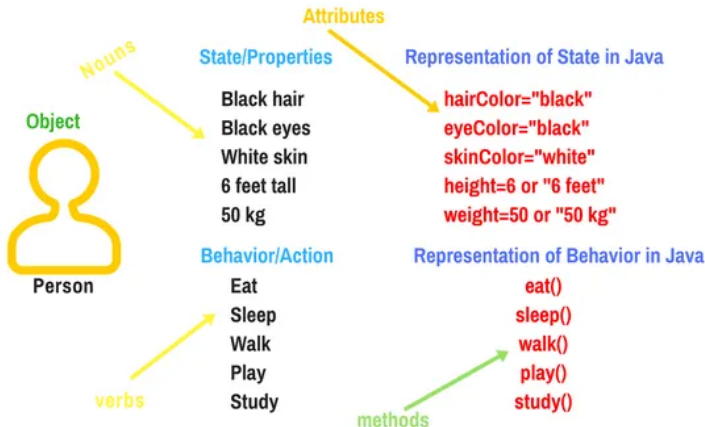
Ce sunt obiectele software?

- Blocurile de construcție a sistemelor software
 - Program = colecție de obiecte care interacționează
 - Obiectele cooperează pentru a finaliza o sarcină; pentru aceasta, ele comunică trimițându-si "mesaje" unul altuia
- Obiectele modelează lucruri tangibile: Persoană, Bicicletă, Cal, Bancă etc.
- Obiectele modelează lucruri conceptuale: întâlnire, dată calendaristică
- Obiectele modelează procese: aflarea drumului printr-un labirint, sortarea unui pachet de cărți de joc

Obiecte

Proprietăți și capacități

- 1 **Proprietăți:** trăsături (caracteristici) care descriu obiectele
- 2 **Capacități:** ce pot face, cum se comportă



Proprietățile obiectului

Proprietățile : determină cum acționează un obiect

- Pot fi constante (nu se schimbă) sau variabile
- Pot fi ele însele obiecte — pot primi mesaje
- Ex. Capacul borcanului cu gem și gemul în sine sunt obiecte

Proprietățile pot fi:

- **attribute:** lucruri care ajută la descrierea unui obiect
- **componente:** lucruri care sunt "parte a" unui obiect
- **asocieri:** lucruri despre care știe un obiect, dar care nu sunt parte a acelui obiect

Proprietățile obiectului

Starea

Stare: colecție a tuturor proprietăților obiectului; se schimbă dacă o proprietate se schimbă

- unele nu se schimbă, d.e. volanul unei mașini
- altele se schimbă, d.e. culoarea mașinii

Exemplu: proprietățile borcanelor cu gem

- **atribute:** culoare, material, miros
- **componente:** capac, container, etichetă
- **asocieri:** un borcan cu gem poate fi asociat cu încăperea în care se află

Capabilitățile obiectelor:

Acțiuni, comportamente

Obiectele au capabilități care le permit să efectueze acțiuni specifice

- obiectele sunt deștepte -ele "știu" cum să facă anumite lucruri
- un obiect face ceva doar dacă un alt obiect îi spune să-și folosească una dintre capabilități
- Capabilitățile pot fi:
 - **constructori**: stabilesc starea inițială a proprietăților obiectului
 - **acțiuni**: modifică proprietățile obiectului
 - **interogări**: furnizează răspunsuri bazate pe proprietățile obiectului

Ex: borcanele cu gem sunt capabile să efectueze acțiuni specifice

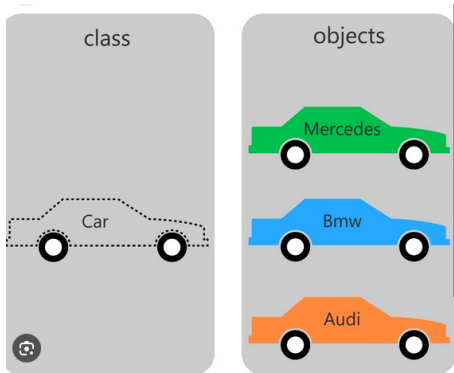
- constructor: să fie creat
- acțiuni: adaugă/golește gem
- interogări: răspunde dacă este închis sau deschis capacul, dacă borcanul este plin sau gol

Clase si instante

Clase și instanțe

- Concepția noastră curentă: fiecare obiect corespunde direct unui anumit obiect din realitate, d.e., un atom sau un automobil anume
- Dezavantaj: mult prea nepractic să lucrăm cu obiecte în acest fel deoarece ele pot fi infinit de multe; nu dorim să descriem fiecare individ separat, deoarece indivizii au multe lucruri în comun

- Clasificarea obiectelor scoate în evidență ce este comun între mulțimi de obiecte similare: mai întâi să descriem ce este comun și apoi să "ștampilăm" oricâte copii



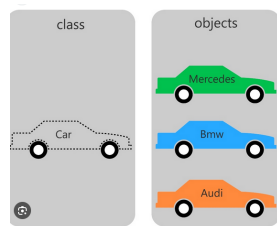
Clase ale obiectelor

Clasa unui obiect: categoria obiectului

- Definește capacitățile și proprietățile comune unei mulțimi de obiecte individuale toate borcanele cu gem se pot deschide, închide și goli
- Definește un șablon pentru crearea de **instanțe de obiect**: unele borcane cu gem pot fi din plastic, pot fi colorate, de o anumită mărime etc.
- Clasele implementează capacitățile ca **metode**:
 - secvențe de instrucțiuni în Java;
 - obiectele cooperează trimițând mesaje altor obiecte;
 - fiecare mesaj "invocă o metodă"
- Clasele implementează proprietățile ca **variabile instanță**: locație de memorie alocată obiectului, care poate păstra o valoare care se poate schimba

Instanțe de obiecte:

- Sunt obiecte individuale realizate din șablonul clasei
- O clasă poate reprezenta un număr nedefinit de instanțe de obiect
- Realizarea unei instanțe de obiect constituie instanțierea obiectului respectiv
- Prescurtare:
 - clasă: clasa obiectului
 - instanță: instanța obiectului (a nu se confunda cu variabilele instanță)
- Instanțe diferite ale clasei Car pot avea culoare diferită, marca diferită etc, astfel că, variabilele instanță ale lor au valori diferite.
- Instanțele individuale au identități individuale: aceasta permite altor obiecte să trimită mesaje unui obiect dat
- Fiecare instanță este unică, chiar dacă are aceleași capacități.



Mesaje pentru comunicarea între obiecte

- Instanțele nu sunt izolate — ele trebuie să comunice cu altele pentru a-și realiza sarcina
 - proprietățile le permit să știe despre alte obiecte
- Instanțele trimit mesaje una alteia pentru a invoca o capacitate (adică, pentru a executa o sarcină)
 - metoda reprezintă codul care implementează mesajul
 - spunem “apelează metoda” în loc de “invocă capacitatea”
- Fiecare mesaj necesită:
 - un emițător (expeditor): obiectul care inițiază acțiunea
 - un receptor: instanța a cărei metode este invocată
 - numele mesajului: numele metodei apelate
 - opțional parametri: informații suplimentare necesitate de metodă pentru a opera
- Receptorul poate (dar nu este nevoit) să trimită un răspuns prin tipurile returnate

Încapsulare

- Un automobil încapsulează multă informație
 - chiar literal, prin complexitatea construcției sale
- Dar nu este nevoie să știi cum funcționează o mașină pentru a o conduce
 - Volanul și schimbarea vitezelor constituie interfața
 - Motorul, transmisia, axul cardanic, roțile, . . . , sunt implementarea (ascunsă)
- Asemănător, nu e nevoie să știm cum funcționează un obiect pentru a-i trimite mesaje
- Dar, este nevoie să știm ce mesaje înțelege (adică, care îi sunt capacitățile): clasa instanței determină ce mesaje îi pot fi trimise
- Închiderea datelor într-un obiect: Datele nu pot fi accesate direct din afară
- Oferă securitatea datelor

Vederile unei clase

Obiectele separă **interfața** de **implementare**:

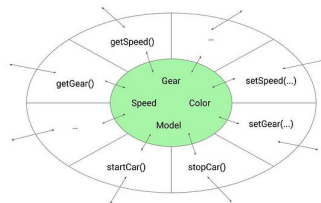
- obiectul este “cutie neagră”; ascunde funcționarea și părțile interne
- interfața protejează implementarea împotriva utilizării greșite

Interfața: vedere publică:

- permite instanțelor să coopereze unele cu altele fără a ști prea multe detalii
- ca un contract: constă dintr-o listă de capacități și documentație pentru cum să fie folosite

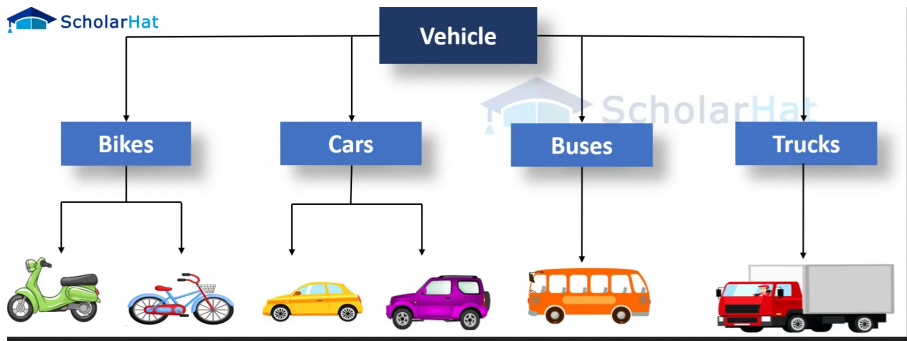
Implementarea: vedere privată

- proprietățile care ajută capacitățile să-și îndeplinească sarcinile



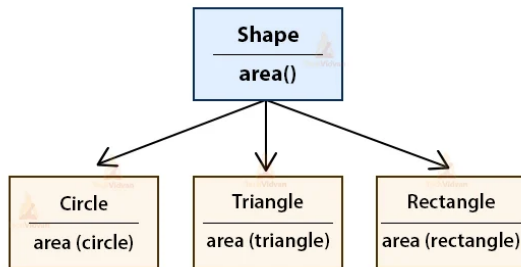
Moștenire

- O clasă (subclasă) poate moșteni atribute și metode dintr-o altă clasă (superclasă)
- Subclasele furnizează comportament specializat
- Oferă reutilizarea codului
- Evită duplicarea datelor



Polimorfism

- Abilitatea de a lua multe forme
- **Aceeași metodă** folosită într-o superclasă poate fi suprascrisă în subclase pentru a da o **funcționalitate diferită**
- D.e. Superclasa 'Shape' are o metodă numită, `area()`:
 - `area()` în subclasa 'Triangle' → $a = x * y / 2$
 - `area()` în subclasa 'Rectangle' → $a = x * y$



Despre JAVA

JAVA

James Gosling și Patrick Naughton (conducătorii echipei care a dezvoltat limbajul), au definit limbajul Java ca fiind:

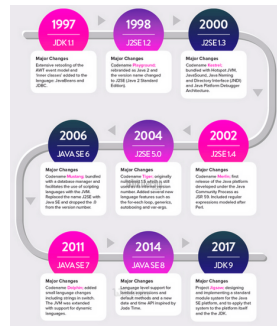
- "Un limbaj simplu, orientat pe obiecte, care "înțelege" rețelele de calculatoare, interpretat, robust, sigur, neutru față de arhitecturi, portabil, de înaltă performanță, multi-fir, dinamic"

Java 8 – Major Release: <https://www.oracle.com/java/technologies/javase/8-whats-new.html>

De ce JAVA 8: Oferă cunoștințe și facilitati de bază ale programării orientate pe obiect;

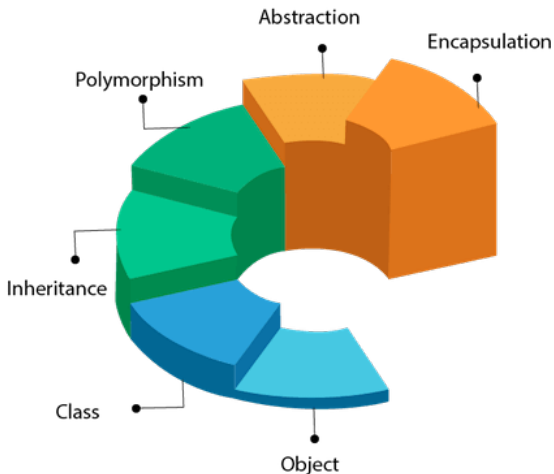
Toate versiunile > 8 adauga diverse trasaturi peste baseline-ul Java 8 ;

La curs și laborator – folosim Java 8 !



Concepte OOP - in JAVA

OOPs (Object-Oriented Programming System)



Executarea programelor JAVA

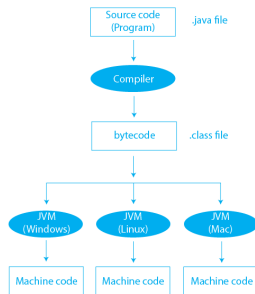
Un proces în doi pași

1 Compilează programul în bytecodes:

- bytecode este apropiat de formatul instrucțiunilor în limbaj mașină, dar nu chiar la fel — este un "limbaj mașină" generic
- nu corespunde nici unui procesor real

2 Mașina virtuală (VM) interpretează bytecode în limbaj mașină nativ și-l rulează

- Există mașini virtuale diferite pentru calculatoare diferite, deoarece bytecode nu corespunde unei mașini reale
- Se utilizează același bytecode Java pe calculatoare diferite fără a recompila codul sursă:
 - Fiecare VM interpretează același bytecode Java; Aceasta permite să se execute programe Java prin simpla obținere a bytecodes din pagini de Web
- Codul Java rulează peste-platforme (cross-platform): "Scrie o dată, rulează oriunde!"



Aplicații JAVA

- 1 Desktop: aplicații cu interfață grafică pentru utilizatori (GUI), care rulează pe sistemul de operare al calculatorului. Ele pot fi dezvoltate cu ajutorul bibliotecilor precum: Swing, JavaFX
- 2 Web: aplicațiile web rulează pe servere și interacționează cu utilizatorii prin intermediul unui browser. Tehnologiile Java utilizate în acest scop includ: Java Servlets, JavaServer Pages (JSP), Spring sau Java EE.
- 3 Mobile: platforma Android este construită pe baza unui subset al Java, iar majoritatea aplicațiilor Android sunt scrise în Java (deși Kotlin devine tot mai popular).
- 4 Distribuite/Enterprise
- 5 Aplicații de tip "microservicii"
- 6 Aplicații pentru Big Data
- 7 Aplicații în Cloud

Aplicație simplă JAVA

- 1 Aplicație de-sine-stătoare sau un program "obișnuit" este o clasă care are o metodă numită main.
 - Când se lansează programul Java respectiv, sistemul de execuție invocă automat metoda numită main
 - Toate aplicațiile de-sine-stătătoare încep din metoda main

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Hello world!");  
4     }  
5 }
```

Aplicație JAVA

Clasa Dreptunghi

Scrieți codul JAVA pentru o aplicație care lucrează cu dreptunghiuri.
Pentru un dreptunghi definit prin lățime și înălțime:

- se citește de la tastatură lățimea și înălțimea,
- se calculează aria și perimetrul,
- se afișează rezultatele.

Aplicație JAVA

Clasa Dreptunghi

```
1  import java.util.Scanner;
2
3  class Dreptunghi {
4      private double latime;
5      private double inaltime;
6
7      // Constructor
8      public Dreptunghi(double latime, double inaltime) {
9          this.latime = latime;
10         this.inaltime = inaltime;
11     }
12
13     public double calculeazaAria() {
14         return latime * inaltime;
15     }
16
17     public double calculeazaPerimetrul() {
18         return 2 * (latime + inaltime);
19     }
20 }
```

Aplicație JAVA

Clasa Dreptunghi

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Scanner sc = new Scanner(System.in);  
4  
5         System.out.print("Introdu latimea: ");  
6         double latime = sc.nextDouble();  
7  
8         System.out.print("Introdu inaltimea: ");  
9         double inaltime = sc.nextDouble();  
10  
11        Dreptunghi d = new Dreptunghi(latime, inaltime);  
12  
13        System.out.println("Aria: " + d.calculeazaAria());  
14        System.out.println("Perimetrul: " + d.calculeazaPerimetrul  
15            ());  
16    }
```

Încărcătorul de clase

Programele Java sunt divizate în unități mai mici numite clase:

- Fiecare definiție de clasă este în mod normal într-un fișier separat și compilată separat

Încărcătorul de clase este un program care leagă bytecode-ul claselor necesare pentru a rula un program Java:

- în alte limbaje de programare, corespondentul său este editorul de legături (link-editor)

Compilarea și rularea unui program JAVA

Compilarea: fiecare definiție de clasă trebuie să se afle într-un fișier al cărui nume este numele clasei urmat de extensia .java:

- Ex: clasa UnProgram trebuie să se afle în fișierul numit UnProgram.java

Fiecare clasă este compilată folosind comanda javac urmată de numele fișierului care conține clasa:

`javac UnProgram.java`

- Rezultatul este un program în byte-code cu același nume ca al clasei, urmat de extensia .class UnProgram.class

Rularea: Un program Java poate fi rulat după ce i-au fost compilate toate clasele

- Rulați doar clasa care conține o metodă main (sistemul va încărca și rula celelalte clase automat, dacă mai sunt)
- Metoda main are semnătura **`public static void main(String[ ] args)`**
- Comanda de lansare a programului trebuie urmată doar de numele clasei (fără extensii) **`java UnProgram`**

Cuvinte cheie Java

cuvinte rezervate

Cuvinte care nu pot fi folosite la altceva decât în modul predefinit din limbaj

- abstract, assert, boolean, break, byte, case, catch, char, class, const, continue, default, do, double, else, extends, final, finally, float, for, goto, if, implements, import, instanceof, int, interface, long, native, new, package, private, protected, public, return, short, static, strictfp, super, switch, synchronized, this, throw, throws, transient, try, void, volatile, while
- null, true, false – predefinite ca literali

Convenții pentru nume

- Începeți numele de variabile, metode și obiecte cu o literă mică, indicați limitele "cuvintelor" cu o literă mare și pentru celelalte caractere folosiți doar litere și cifre ("camelcase")

vitezaMaxima ; rataDobanzii ; oraSosirii

- Începeți numele de clase cu majusculă și pentru restul identificatorului aplicați regula de mai sus:

UnProgram ; OClasa ; String

Declararea variabilelor

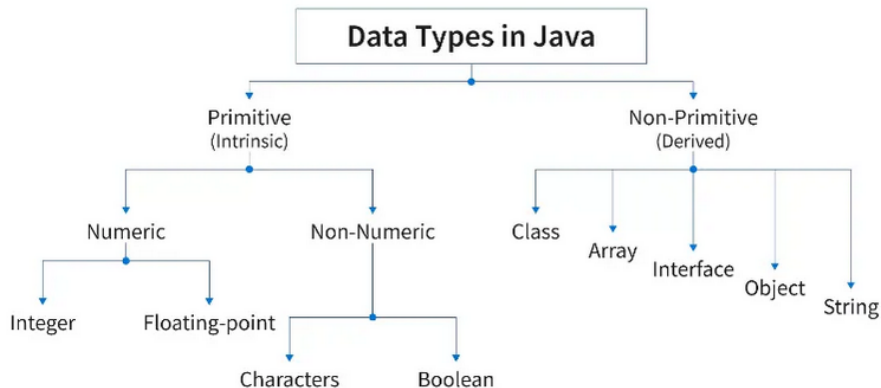
Fiecare variabilă dintr-un program Java trebuie declarată înainte de utilizare:

- Declarația informează compilatorul asupra tipului de dată care va fi stocat în variabilă
- Tipul variabilei este urmat de unul sau mai multe nume separate de virgule și terminat cu punct și virgulă
- Variabilele se declară de obicei chiar înainte de folosire sau la începutul unui bloc (indicat de o acoladă deschisă {)
- Tipurile simple în Java sunt numite tipuri primitive:
 - `int varsta;`
 - `double oLungime, lungimeaTotala;`

Modificatori de acces pentru variabile și metode

- ❶ **private:** variabila/metoda este vizibilă local în cadrul clasei ("Vizibilă doar mie")
- ❷ **protected:** variabila/metoda poate fi văzută din toate clasele, subclasele și celelalte clase din același pachet (package) ("Vizibilă în familie")
- ❸ **public:** variabila/metoda poate fi văzută din toate clasele ("Vizibilă tuturor")
- ❹ Modificatorul de acces **implicit, nu are cuvânt cheie:**
 - public pentru ceilalți membrii din același pachet
 - private pentru oricine din afara pachetului
 - numită și acces în pachet ("Vizibilă în vecinătate")

Tipuri de date în JAVA



Tipuri de date primitive în JAVA

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\l', '\', '\n', '\b'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Tipuri de date primitive în JAVA

Exemplu

```
1 public class Main {  
2     public static void main(String[] args) {  
3         int myNum = 9000;  
4         System.out.println(myNum);  
5         double myNum1 = 29.99d;  
6         System.out.println(myNum1);  
7         char myGrade = 'B';  
8         System.out.println(myGrade);  
9         String greeting = "Hello World";  
10        System.out.println(greeting);  
11        boolean isNone = true;  
12        System.out.println(isNone);  
13    }  
14 }
```

Compatibilitatea la asignare

- O valoare de orice tip din lista următoare poate fi asignată unei variabile de orice alt tip care apare la dreapta ei (Gama valorilor de la dreapta este mai largă)
- *byte* → *short* → *int* → *long* → *float* → *double*
- Este necesară o conversie de tip explicită (type cast) pentru a asigna o valoare de un tip la o variabilă care apare la stânga ei în lista de mai sus (d.e., double la int)
- Observați că în Java un int nu poate fi asignat la o variabilă de tip boolean, nici un boolean la o variabilă de tip int

Operatori aritmetici și expresii

- Ca în majoritatea limbajelor, și în Java se pot forma expresii folosind variabile, constante și operatori aritmetici
- Operatori aritmetici; + (adunare), - (scădere), * (înmulțire), / (împărțire), % (modulo, rest)
- Se poate folosi o expresie oriunde este legal să se folosească o valoare de tipul produs de expresie
- Dacă se combină un operator aritmetic cu operanzi de tipul int, atunci tipul rezultat este int
- Dacă se combină un operator aritmetic cu unul sau doi operanzi de tipul double, atunci tipul rezultat este double
- La combinarea de operanzi de tip diferit, tipul rezultat este cel mai din dreapta din lista de mai jos care se află în expresie:
byte → short → int → long → float → double
- Excepție: Dacă tipul rezultat este byte sau short (potrivit regulii date), atunci tipul produs va fi de fapt un int

Reguli de precedență și asociativitate

- La determinarea ordinii operațiilor adiacente, operația cu precedență mai mare (și argumentele sale aparente) este grupată înaintea operației de precedență mai mică:

`base + rate * hours`

se evaluează ca `base + (rate * hours)`

- La precedență egală, ordinea operațiilor este determinată de regulile de asociativitate

Numere reale, reprezentare în virgula mobilă, împărțirea întreagă și cea în virgulă mobilă

Numerele în virgulă mobilă sunt, în general, doar valori aproximative

- Matematic, numărul în virgulă mobilă $1.0/3.0$ este egal cu $0.3333333\ldots$
- Un calculator are o cantitate limitată de memorie, poate stoca $1.0/3.0$ ca ceva în genul lui 0.3333333333 , puțin mai puțin decât o treime
- De fapt numerele sunt stocate binar, dar consecințele sunt aceleași: numerele în virgulă mobilă pot pierde precizie
- Dacă unul sau amândoi operandii sunt în virgulă mobilă, împărțirea dă un rezultat în virgulă mobilă: $15.0/2$ se evaluează la 7.5
- Cu ambii operanzi întregi, împărțirea dă un întreg (o eventuală parte fracționară este ignorată, nu se fac rotunjiri): $15/2$ se evaluează la 7
- Cel puțin un operand trebuie să fie în virgulă mobilă dacă este nevoie de partea fracționară

Conversia de tip explicită

O conversie de tip explicită (type cast) ia o valoare de un tip și produce o valoare "echivalentă" de celălalt tip:

- Dacă n și m sunt întregii de împărțit și e nevoie de partea fracționară, atunci cel puțin un operand trebuie să fie în virgulă mobilă înainte de efectuarea operației: `double ans = n / (double)m;`
- La fel ca în C, tipul dorit este pus între paranteze imediat înaintea variabilei de convertit
- Tipul și valoarea variabilei de convertit nu se schimbă
- La conversia explicită de la virgulă mobilă la întreg, numărul este trunchiat, nu rotunjit: `(int)2.9` se evaluează la 2, nu 3
- La asignarea valorii unui întreg la o variabilă în virgulă mobilă, Java realizează o conversie explicită de tip automată numită coerciție de tip: `double d = 5;`
- Nu este legal să se atribuie un `double` la un `int` fără o conversie explicită:
 - `int i = 5.5; // Illegal`
 - `int i = (int)5.5 // Corect`

Operatorii increment și decrement

- Când oricare dintre operatorii $++$ sau $--$ precede o variabilă și este o parte a expresiei, expresia este evaluată folosind valoarea modificată a variabilei
- Dacă n este 2, atunci $2*(++n)$ se evaluează la 6
- Când oricare dintre operatori urmează unei variabile și este parte a expresiei, expresia este evaluată folosind valoarea originală și abia apoi se schimbă valoarea variabilei
- Dacă n este 2, atunci $2*(n++)$ se evaluează la 4

Tipuri de date referință în JAVA

Tipuri de date non-primitive

- Arrays:

```
1 String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
2 int[] myNum = {10, 20, 30, 40};
3 for (String i : cars) {
4     System.out.println(i);
5 }
```

- Clase / Obiecte

```
1 public class Main {
2     int x = 7;
3
4     public static void main(String[] args) {
5         Main myObj = new Main();
6         System.out.println(myObj.x);
7     }
8 }
```

Concluzii